Bader's Event Managing System

User & Developer Documentation

A Java console application for creating, scheduling, and reviewing campus events — with automatic time-clash detection. This document explains how to run the program, what each command does, and how the underlying classes and methods fit together.

Quick Facts

Property	Value
Language	Java (no external libraries)
Entry point	Main.java
Input modes	Text file • Manual (keyboard)
Core feature	Auto-prevents events with overlapping times
Storage	In-memory (data resets when the program quits)

Contents

- 1. Overview
- 2. Getting Started — Compile & Run
- 3. The Two Input Modes
- 4. Menu Commands
- 5. Creating an Event — Step by Step
- 6. Input Format Reference
- 7. Worked Example
- 8. Class Overview
- 9. Key Methods Reference
- 10. Errors, Warnings & Edge Cases
- 11. Tips & Known Issues

1. Overview

Bader's Event Managing System is a small console program that lets the user schedule events on a campus. Every event has a name, a start date/time, an end date/time, a responsible department, and a venue. The program keeps the events sorted in chronological order automatically and refuses to add a new event whose time window overlaps with one that already exists.

What you can do with it

- **Create** a new event by entering its details.
- **List** all events in a clean table sorted by start time.
- **Inspect** a single event in detail (department, venue, capacity, etc.).
- **Quit** when you're done.

2. Getting Started — Compile & Run

All `.java` files must sit in the same folder. Open a terminal in that folder and run:

```
javac *.java
java Main
```

If you also have a test input file (for example `Test.txt`), open `Main.java` and set the file name on this line:

```
String fileName = "[FILE_NAME_GOES_HERE]";
```

Replace the placeholder with the path of your file, e.g. `"Test.txt"`. If the file is missing or the name is still the placeholder, the program prints an error and falls back to manual entry from the keyboard — so it always runs either way.

3. The Two Input Modes

The program has a single **showMenu** flag that controls how chatty it is:

Mode	When it activates	Behavior
File mode	The configured file is found and opened.	Reads commands silently. No prompts are printed.
Manual mode	The file is missing, so the program falls back.	Prints a menu and prompts before each input.

Important: in file mode you will not see any prompts — but the program still expects exactly the same sequence of inputs as in manual mode, one per line.

4. Menu Commands

Command	Meaning	Notes
C	Create a new event	Triggers the 11-line input flow shown in Section 5.
SE	Show all events	Prints a sorted table with index, name, start, end.
SI	Show one event in detail	Prompts for an index (0 = first event).
Q	Quit the program	Always works, in both modes.

Commands are case-sensitive. Type `C`, not `c`.

5. Creating an Event — Step by Step

After you type `C`, the program asks for the following 11 inputs in this exact order, one per line:

#	Field	Example
1	Event name	Aramco Football
2	Start date — DD MM YYYY	1 1 2000
3	End date — DD MM YYYY	2 1 2000
4	Start time — HH MM	14 00
5	End time — HH MM	16 00
6	Department name	Sports
7	Person in charge	Ahmed Al-Saud
8	Venue name	Main Stadium
9	Venue info	Outdoor field with seating
10	Maximum capacity (int)	5000

> **Watch out:** dates and times are entered with a single space separator (`1 1 2000`, not `1/1/2000`). Capacity must be a whole number with no decimal point.

6. Input Format Reference

Date format

```
DD MM YYYY     // example: 5 3 2000  -> 5 March 2000
Validation rules:
  - month must be 1..12
  - year must be >= 1
  - day must fit the month (leap years are handled in February)
```

Time format

```
HH MM          // 24-hour clock, example: 14 00  -> 2:00 PM
Validation rules:
  - hour must be 0..23
  - minutes must be 0..59
```

If any of these rules fail, `DateAndTime` throws an `IllegalArgumentException` and the program stops. There is currently no retry loop, so make sure the values are valid.

7. Worked Example

Here is a complete session — exactly the contents of the bundled `Test.txt`. It creates three events, lists them, shows the details of event #1, and quits:

```
C
Aramco Football
1 1 2000
2 1 2000
14 00
16 00
Sports
Ahmed Al-Saud
Main Stadium
Outdoor football field with seating
5000
C
KFUPM Hackathon
5 3 2000
6 3 2000
09 00
18 00
Computer Science
Dr. Khalid Al-Otaibi
Building 22 Auditorium
Conference hall with tables and projectors
200
C
Volleyball Tournament
10 4 2000
10 4 2000
17 00
19 00
Recreation
Faisal Al-Rashid
Sports Center
Indoor volleyball court with bleachers
800
SE
SI
1
Q
```

After SE, expect output similar to:

```
----------------------------------------------------------------------
|                        List Of Events                              |
----------------------------------------------------------------------
| 0 | Aramco Football         | 01/01/2000  14:00 | 02/01/2000  16:00 |
| 1 | KFUPM Hackathon         | 05/03/2000  09:00 | 06/03/2000  18:00 |
| 2 | Volleyball Tournament   | 10/04/2000  17:00 | 10/04/2000  19:00 |
----------------------------------------------------------------------
```

8. Class Overview

Class	Role
Main	Reads commands, drives the menu loop, builds an Event from user input.
Event	A plain data holder: name, start, end, department, venue.
EventManager	Stores the events in a sorted ArrayList; checks for clashes; prints tables.
DateAndTime	Validated date+time. Implements Comparable so events can be sorted.
Department	Holds a department's name and the person responsible.
DepartmentsManager	Abstract class with a static catalog of pre-defined departments.
Venue	Holds a venue's name, description, and capacity.
VenueManager	Abstract class with a static catalog of pre-defined venues.

9. Key Methods Reference

DateAndTime

Constructor — `DateAndTime(int day, int month, int year, int minutes, int hour)`

Validates the date (using `isValidDate`) and the time. Throws `IllegalArgumentException` for any out-of-range value. Note the parameter order: minutes comes *before* hour.

compareTo(DateAndTime other)

Returns a negative number if *this* is earlier, 0 if equal, positive if later. Compares year, then month, then day, then hour, then minute. This is what makes events sortable.

isValidDate(int d, int m, int y) · daysInMonth(int m, int y)

Static helpers. `daysInMonth` returns 28 or 29 for February depending on whether the year is a leap year, 30 or 31 for the rest, or –1 if the month is invalid.

EventManager

addEvent(Event event)

First calls `isFree`. If the new event clashes with an existing one, it prints a warning and the event is *not* added. Otherwise it walks the list and inserts the event at the position that keeps the list sorted by start time.

isFree(Event other)

Returns `true` when no stored event overlaps the given one. Two events overlap when one's end is strictly after the other's start *and* the other's start is strictly before the first one's end. Back-to-back events (end == start) are allowed.

showEvents()

Prints a bordered table with the index, name, start, and end of every stored event, in chronological order.

showEventInfo(int index) · showEventInfo(Event event)

Overloaded. The *int* version looks the event up by its position in the table and forwards to the *Event* version, which prints a detailed card including department, person in charge, venue, venue info, and capacity.

Department & Venue

Both are simple data classes with getters: `getName()`, `getPerson()` (Department), `getInfo()`, `getCapacity()` (Venue).

DepartmentsManager & VenueManager

Both are `abstract` with static fields and methods — they're never instantiated. They hold predefined catalogs:

- **DepartmentsManager.availableDepartments**: ICS, COE, ISE.
- **VenueManager.availableVenues**: Sport Area, Lecture Hall, Conference Hall, Public Space.
- **DepartmentsManager.getDepartment(String)**: looks up a department by name; throws `IllegalArgumentException` if not found.

> **Heads-up:** the current `Main` doesn't actually consult these catalogs — it builds a fresh `Department` and `Venue` from whatever the user types. The catalogs are ready to use if you wire them in later (see Section 11).

10. Errors, Warnings & Edge Cases

Situation	What you'll see
File name is missing or wrong	An [ERROR] line, then it switches to manual entry.
Date or time is out of range	IllegalArgumentException — the program stops.
New event overlaps an existing one	Message: "Couldn't Add Event ... Due To Date And Time Clash". Event is skipped.
SI command with a bad index	Error message about the valid range, then no detail card is shown.
Capacity is not a whole number	NumberFormatException — the program stops.
End is before Start	Currently allowed; the program does not check this.

11. Tips & Known Issues

- **Set the file name before you run.** The default value `[FILE_NAME_GOES_HERE]` is a placeholder; replace it with `"Test.txt"` (or the path of your real file) to use file mode.
- **Menu typo.** The text printed by `printMenu()` says "show the events" for both `SE` and `SI`; the second one actually shows the details of one event. Worth fixing the message.
- **Hook up the catalogs.** If you want users to pick from the predefined departments and venues instead of typing them in freely, replace the manual `new Department(...)` / `new Venue(...)` calls in `Main` with calls like `DepartmentsManager.getDepartment(name)`.
- **End-before-start.** The constructor doesn't check that the end is after the start. Adding a single comparison in the `Event` constructor would close that gap.
- **Persistence.** Events live only while the program is running. To keep them between runs, write them to a file on quit and reload them on startup.
- **Case sensitivity.** Menu commands are matched with `equals`, so `q` and `Q` are different. Use the uppercase form.

End of document.